

Introduction to Probabilistic Machine Learning

Non-Bayesian Classification

Ralf Herbrich

Overview

1. Geometry of Linear Classifiers
2. Fisher's Linear Discriminant
3. Perceptron Learning Algorithm
4. Logistic Regression

**Introduction to
Probabilistic Machine
Learning**

*Unit 11 – Non-Bayesian
Classification*

Overview

- 1. Classification Learning**
2. Fisher's Linear Discriminant
3. Perceptron Learning Algorithm
4. Logistic Regression

**Introduction to
Probabilistic Machine
Learning**

*Unit 11 – Non-Bayesian
Classification*

Classification Learning Setting

■ Given:

1. **Training Data:** $D \in (\mathcal{X} \times \{0,1\})^n$ of n labelled examples (x, y) from input space $\mathcal{X}$
2. **Linear Basis Functions:** Basis function mapping $\phi: \mathcal{X} \rightarrow \mathbb{R}^M$ and linear function model

$$f(x; \mathbf{w}) := \mathbf{w}^T \phi(x)$$

3. **Likelihood of functions:** Let $g: \mathbb{R} \rightarrow [0,1]$ be a fixed monotone function. Then

$$p(D|f) = p(D|\mathbf{w}) = \prod_{i=1}^n \text{Ber}(y_i; g(\mathbf{w}^T \phi(x_i)))$$

4. **Prior belief over functions:**

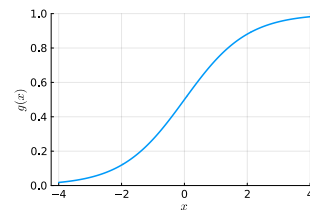
$$p(f) = p(\mathbf{w}) = \prod_{j=1}^M \mathcal{N}(w_j; 0, \lambda^2)$$

■ Bayesian Inference:

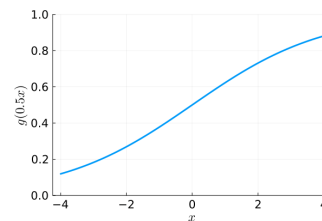
- **Posterior belief over functions:**

$$p(f|D) = p(\mathbf{w}|D) = \frac{\prod_{i=1}^n \text{Ber}(y_i; g(\mathbf{w}^T \phi(x_i))) \cdot \prod_{j=1}^M \mathcal{N}(w_j; 0, \lambda^2)}{\int_{\mathbb{R}^M} \prod_{i=1}^n \text{Ber}(y_i; g(\tilde{\mathbf{w}}^T \phi(x_i))) \cdot \prod_{j=1}^M \mathcal{N}(\tilde{w}_j; 0, \lambda^2) d\tilde{\mathbf{w}}}$$

$$g(t) = \frac{\exp(t)}{1 + \exp(t)}$$



$$\lim_{s \rightarrow \infty} g(s \cdot t) = \mathbb{I}(t > 0)$$

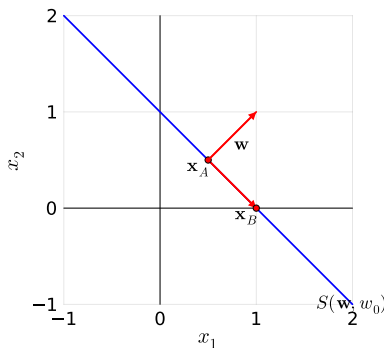


Geometry of Linear Classifiers

- **Linear Model:** Consider the case where $\phi_0 = 1$, $\phi_i(x) = x_i$ and $\lim_{s \rightarrow \infty} g(s \cdot t) = \mathbb{I}(t > 0)$.

$$f(x; w, w_0) = w^T x + w_0$$

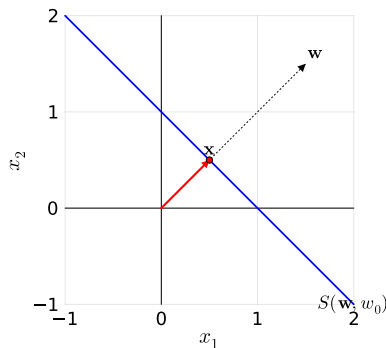
- **Decision Surface.** The decision surface $S(w, w_0) \subseteq \mathcal{X}$ are all points x where $f(x; w, w_0) = 0$.



$$w^T x_A + w_0 = 0$$

$$w^T x_B + w_0 = 0$$

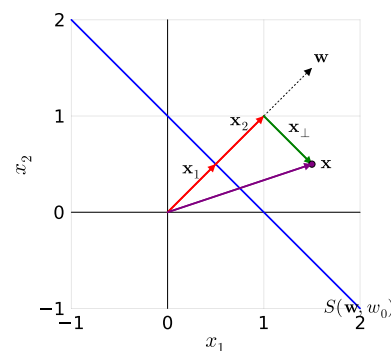
$$w^T (x_B - x_A) = 0$$



$$x = \alpha w$$

$$w^T x + w_0 = \alpha w^T w + w_0 = 0$$

$$\|x\| = \alpha \|w\| = -\frac{w_0}{\|w\|^2} \cdot \|w\| = -\frac{w_0}{\|w\|}$$



to
achieve

$$x = x_1 + x_2 + x_\perp = (\alpha + r)w + x_\perp$$

$$w^T x = (\alpha + r)\|w\|^2 \Leftrightarrow r = \frac{w^T x + w_0}{\|w\|^2}$$

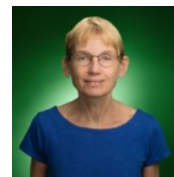
$$\|x_2\| = r \|w\| = \frac{w^T x + w_0}{\|w\|}$$

Classification Learning Example: MNIST

- **MNIST** (Modified National Institute of Standards and Technology database) is a database of handwritten digits
 - Created in 1998 from two datasets of handwritten digits of high-school students and American Census Bureau employees
 - Created by Yann LeCun, Corinna Cortes, and Christ Burges
 - 60,000 training images/10,000 test images
 - Original black-and-white images were scaled to 28x28 pixels and anti-aliased to give grey scales
- Standard dataset to compare classification learning algorithms from 1998 – 2010 (before ImageNet)
- **Smallest prediction error: 0.18% (2018)**



Yann LeCun
(1960 –)



Corinna Cortes
(1961 –)



Chris Burges
(1960 –)

**Introduction to
Probabilistic Machine
Learning**

*Unit 11 – Non-Bayesian
Classification*

Overview

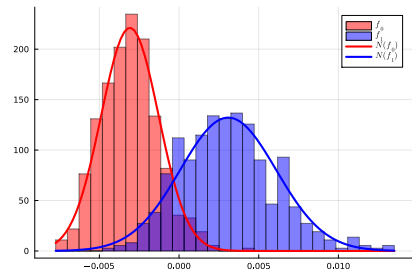
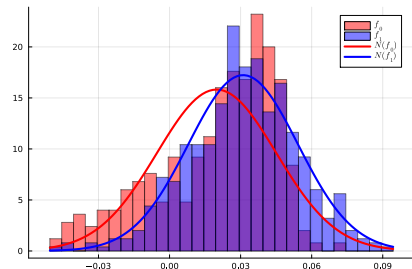
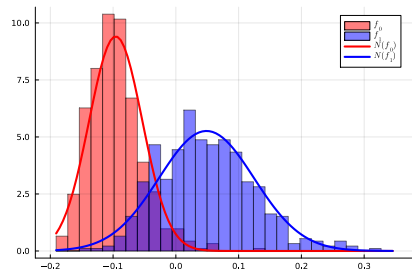
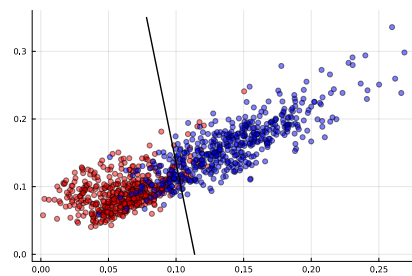
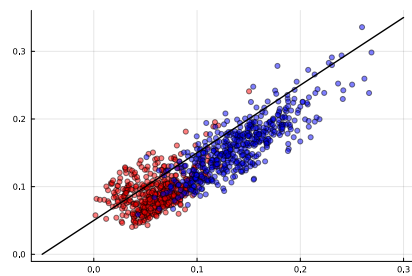
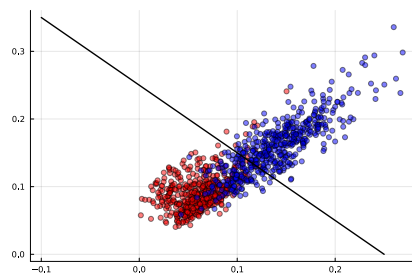
1. Classification Learning
- 2. Fisher's Linear Discriminant**
3. Perceptron Learning Algorithm
4. Logistic Regression

**Introduction to
Probabilistic Machine
Learning**

*Unit 11 – Non-Bayesian
Classification*

Marginal Distribution after Linear Projection

- **Idea:** Given a linear basis function model $f(x; \mathbf{w}, w_0) := \mathbf{w}^T \boldsymbol{\phi}(x) + w_0$, estimate the marginal distribution of the $f(x_i)$ with a Normal distribution separately for the class $y_i = 0$ and $y_i = 1$



Introduction to Probabilistic Machine Learning

Unit 11 – Non-Bayesian Classification

Fisher's Criterion

- **Distribution of projection** per class k : $p(f(x; \mathbf{w}, w_0) | \mathbf{w}, w_0, y = k) \approx \mathcal{N}(f; \mu_k, \sigma_k^2)$

$$\mu_{\text{ML},k}(\mathbf{w}) = \frac{1}{n_k} \sum_{y_i=k} \mathbf{w}^T \boldsymbol{\phi}(x_i) + w_0$$

$$\sigma_{\text{ML},k}^2(\mathbf{w}) = \frac{1}{n_k} \sum_{y_i=k} \left(\mathbf{w}^T \boldsymbol{\phi}(x_i) - \mu_{\text{ML},k}(\mathbf{w}) \right)^2$$

- **Fisher Criterion:** Maximize over $\mathbf{w}$ *between-class* distance and minimize *within-class* distance

$$J(\mathbf{w}) = \frac{[\mu_{\text{ML},0}(\mathbf{w}) - \mu_{\text{ML},1}(\mathbf{w})]^2}{\sigma_{\text{ML},0}^2(\mathbf{w}) + \sigma_{\text{ML},1}^2(\mathbf{w})}$$

$$\mathbf{m}_k := \frac{1}{n_k} \sum_{y_i=k} \boldsymbol{\phi}(x_i)$$

$$J(\mathbf{w}) = \frac{[\mathbf{w}^T(\mathbf{m}_0 - \mathbf{m}_1)]^2}{\frac{1}{n_0} \sum_{y_i=0} [\mathbf{w}^T(\boldsymbol{\phi}(x_i) - \mathbf{m}_0)]^2 + \frac{1}{n_1} \sum_{y_i=1} [\mathbf{w}^T(\boldsymbol{\phi}(x_i) - \mathbf{m}_1)]^2}$$

↖ w_0 cancels



Sir Ronald Fisher
(1890 – 1962)

Introduction to
Probabilistic Machine
Learning

Unit 11 – Non-Bayesian
Classification

Fisher's Linear Discriminant Analysis (LDA)

- **Expanding the Fisher Criterion** by introducing matrices S_B and S_W

$$J(\mathbf{w}) = \frac{\mathbf{w}^T \overbrace{(\mathbf{m}_0 - \mathbf{m}_1)(\mathbf{m}_0 - \mathbf{m}_1)^T}^{S_B} \mathbf{w}}{\underbrace{\mathbf{w}^T \left(\frac{1}{n_0} \sum_{y_i=0} (\boldsymbol{\phi}(x_i) - \mathbf{m}_0)(\boldsymbol{\phi}(x_i) - \mathbf{m}_0)^T + \frac{1}{n_1} \sum_{y_i=1} (\boldsymbol{\phi}(x_i) - \mathbf{m}_1)(\boldsymbol{\phi}(x_i) - \mathbf{m}_1)^T \right) \mathbf{w}}_{S_W}}$$

- **Solution:** Maximize $J(\mathbf{w})$ over the choice of $\mathbf{w}$.

$$\frac{\partial J(\mathbf{w})}{\partial \mathbf{w}} = \frac{\partial}{\partial \mathbf{w}} \underbrace{\frac{\mathbf{w}^T S_B \mathbf{w}}{\mathbf{w}^T S_W \mathbf{w}}}_{\text{Rayleigh coefficient}}$$

$$\left. \frac{\partial J(\mathbf{w})}{\partial \mathbf{w}} \right|_{\mathbf{w}_{\text{LDA}}} = 2S_B \mathbf{w}_{\text{LDA}} \cdot (\mathbf{w}_{\text{LDA}}^T S_W \mathbf{w}_{\text{LDA}})^{-1} - 2(\mathbf{w}_{\text{LDA}}^T S_B \mathbf{w}_{\text{LDA}})(\mathbf{w}_{\text{LDA}}^T S_W \mathbf{w}_{\text{LDA}})^{-2} S_W \mathbf{w}_{\text{LDA}} = 0$$

$$S_W \mathbf{w}_{\text{LDA}} = \frac{\mathbf{w}_{\text{LDA}}^T S_W \mathbf{w}_{\text{LDA}}}{\mathbf{w}_{\text{LDA}}^T S_B \mathbf{w}_{\text{LDA}}} S_B \mathbf{w}_{\text{LDA}}$$

$$S_W \mathbf{w}_{\text{LDA}} \propto (\mathbf{m}_0 - \mathbf{m}_1) [(\mathbf{m}_0 - \mathbf{m}_1)^T \mathbf{w}_{\text{LDA}}] \propto (\mathbf{m}_0 - \mathbf{m}_1)$$

$$\mathbf{w}_{\text{LDA}} \propto S_W^{-1} (\mathbf{m}_0 - \mathbf{m}_1)$$

**Introduction to
Probabilistic Machine
Learning**

Unit 11 – Non-Bayesian
Classification

Fisher's Linear Discriminant Analysis (ctd.)

- **Question:** How to determine the **optimal threshold** w_0 ?
- **Idea:** Choose it such that the weighted mid-point between the two means $\mu_{\text{ML},0}(\mathbf{w}_{\text{LDA}})$ and $\mu_{\text{ML},1}(\mathbf{w}_{\text{LDA}})$ is exactly at zero (remember $g(x) = \mathbb{I}(x > 0)$)!

$$\frac{n_0}{n_0 + n_1} \cdot \mu_{\text{ML},0}(\mathbf{w}_{\text{LDA}}) + \frac{n_1}{n_0 + n_1} \cdot \mu_{\text{ML},1}(\mathbf{w}_{\text{LDA}}) = 0$$

$$\frac{n_0}{n_0 + n_1} \left(\frac{1}{n_0} \sum_{y_i=0} \mathbf{w}_{\text{LDA}}^T \boldsymbol{\phi}(x_i) + w_{\text{LDA},0} \right) + \frac{n_1}{n_0 + n_1} \left(\frac{1}{n_1} \sum_{y_i=1} \mathbf{w}_{\text{LDA}}^T \boldsymbol{\phi}(x_i) + w_{\text{LDA},0} \right) = 0$$

$$\underbrace{\mathbf{w}^T \left(\frac{1}{n_0 + n_1} \sum_{i=1}^{n_0+n_1} \boldsymbol{\phi}(x_i) \right)}_m + w_{\text{LDA},0} = 0$$

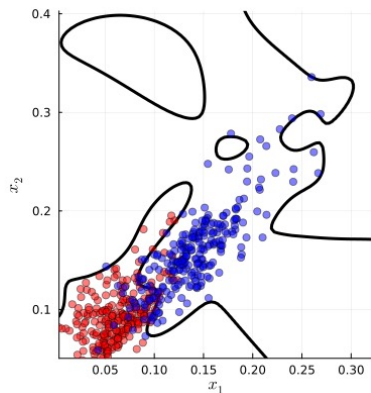
$$w_{\text{LDA},0} = -\mathbf{w}_{\text{LDA}}^T m$$

**Introduction to
Probabilistic Machine
Learning**

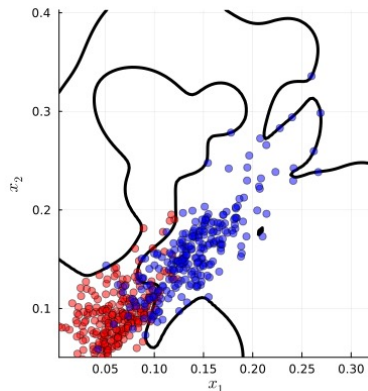
*Unit 11 – Non-Bayesian
Classification*

Fisher's Linear Discriminant Analysis in Pictures

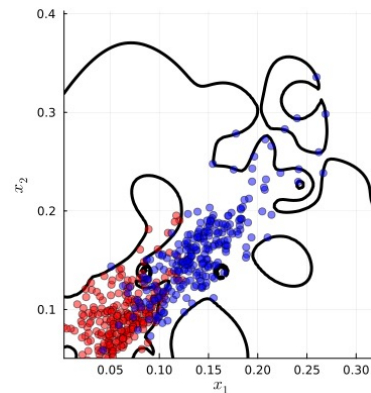
3 variances



5 variances



9 variances



Feature values of Gaussian density in 2D on a 5×5 grid at n decaying variances

**Introduction to
Probabilistic Machine
Learning**

*Unit 11 – Non-Bayesian
Classification*

Overview

1. Classification Learning
2. Fisher's Linear Discriminant
- 3. Perceptron Learning Algorithm**
4. Logistic Regression

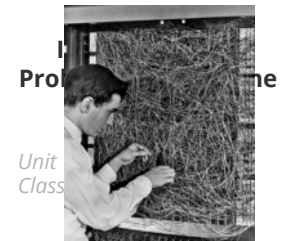
**Introduction to
Probabilistic Machine
Learning**

*Unit 11 – Non-Bayesian
Classification*

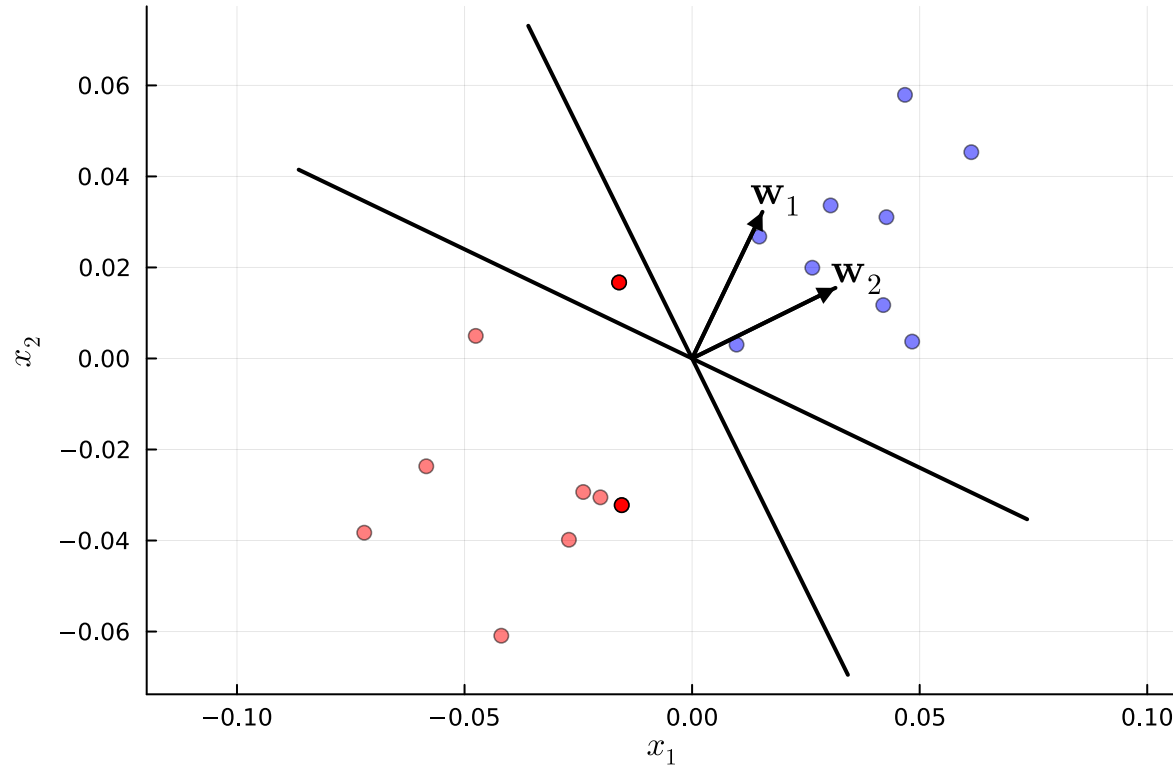
Perceptron Learning

- **Perceptron:** First model of a classification system based on neural activity
 - Invented in 1943 by McCulloch and Pitts.
 - Implemented as a machine by Frank Rosenblatt at Cornell in 1958
 - Was intended to be a “machine”, not an algorithm (!), built on the IBM Mark 1
 - It had an array of 400 photocells
 - Weights were implemented by potentiometers; updates by electric motors
 - Only capable of learning linearly separable datasets
- **Perceptron Algorithm:** One of the simplest classification learning algorithms
 - **Given:** A data set $\{(x_1, y_1), \dots, (x_n, y_n)\} \subseteq (\mathcal{X} \times \{-1, 1\})$

1. **Initialize:** A weight vector $\mathbf{w} = \mathbf{0}$
2. **Iterate** through all examples and **if** $y_i \mathbf{w}^T \boldsymbol{\phi}(x_i) \leq 0$ **then** $\mathbf{w} \leftarrow \mathbf{w} + y_i \boldsymbol{\phi}(x_i)$
3. **Stop** when no mistakes were made on all data



Perceptron Algorithm in Action



**Introduction to
Probabilistic Machine
Learning**

*Unit 11 – Non-Bayesian
Classification*

- $$(\mathbf{a}^T \mathbf{b})^2 \leq \|\mathbf{a}\|^2 \cdot \|\mathbf{b}\|^2$$

- $$0 \leq \|\mathbf{a} + c\mathbf{b}\|^2 = \|\mathbf{a}\|^2 + 2c\mathbf{a}^T\mathbf{b} + c^2\|\mathbf{b}\|^2 = \|\mathbf{a}\|^2 - (\mathbf{a}^T\mathbf{b})^2/\|\mathbf{b}\|^2$$

- margin $\gamma = \max_{\mathbf{w}} \min_{i \in \{1, \dots, n\}} \frac{y_i \mathbf{w}^T \mathbf{x}_i}{\|\mathbf{w}\|}$ the perceptron learning algorithm converges after at most $(R/\gamma)^2$ mistake updates where $R = \max_{i \in \{1, \dots, n\}} \|\mathbf{x}_i\|$.

- $$\mathbf{w}_t^T \mathbf{w}^* = (\mathbf{w}_{t-1} + y_i \mathbf{x}_i)^T \mathbf{w}^* = \mathbf{w}_{t-1}^T \mathbf{w}^* + y_i \mathbf{x}_i^T \mathbf{w}^* \geq \mathbf{w}_{t-1}^T \mathbf{w}^* + \gamma \cdot \|\mathbf{w}^*\| \geq \dots \geq t \cdot \gamma \cdot \|\mathbf{w}^*\|$$

$$\|w_t\|^2 = \|w_{t-1}\|^2 + 2y_i x_i^T w_{t-1} + \|x_i\|^2 \leq \|w_{t-1}\|^2 + R^2 \leq \dots \leq t \cdot R^2$$

because $R = \max_{i \in \{1, \dots, n\}} \|x_i\|$

$$t^2 \cdot \gamma^2 \cdot \|\mathbf{w}^*\|^2 \leq (\mathbf{w}_t^\top \mathbf{w}^*)^2 \leq \|\mathbf{w}_t\|^2 \cdot \|\mathbf{w}^*\|^2 \leq t \cdot R^2 \cdot \|\mathbf{w}^*\|^2$$



Unit 11 – Non-Bayesian Classification

Overview

1. Classification Learning
2. Fisher's Linear Discriminant
3. Perceptron Learning Algorithm
- 4. Logistic Regression**

**Introduction to
Probabilistic Machine
Learning**

*Unit 11 – Non-Bayesian
Classification*

Logistic Likelihood

- **Logistic Likelihood:** Given a dataset $D \in (\mathcal{X} \times \{0,1\})^n$ of n labelled examples from input space $\mathcal{X}$, the basis function mapping $\phi: \mathcal{X} \rightarrow \mathbb{R}^M$ and the logistic sigmoid function $g(t) := \frac{\exp(t)}{1+\exp(t)}$, the logistic likelihood is given by

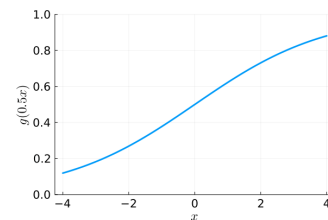
$$P(\mathbf{y}|X, \mathbf{w}) = \prod_{i=1}^n g(\mathbf{w}^T \phi(x_i))^{y_i} \cdot [1 - g(\mathbf{w}^T \phi(x_i))]^{1-y_i}$$

- Taking the **negative logarithm** results in a *cross-entropy* between the empirical class distribution and the predicted class distribution

$$-\log_2 P(\mathbf{y}|X, \mathbf{w}) = -\sum_{i=1}^n \left[y_i \log_2 \left(g(\mathbf{w}^T \phi(x_i)) \right) + (1 - y_i) \log_2 \left(1 - g(\mathbf{w}^T \phi(x_i)) \right) \right]$$

- **Maximum Likelihood Approach:** Find $\mathbf{w}$ which minimizes $-\log_2(P(\mathbf{y}|X, \mathbf{w}))$
 - No unique solution for linearly separable data without scale constraint on $\mathbf{w}$ because
 - There exists many $\tilde{\mathbf{w}}$ with $y_i g(\tilde{\mathbf{w}}^T \phi(x_i)) + (1 - y_i) \log_2(1 - g(\tilde{\mathbf{w}}^T \phi(x_i))) > 0.5$
 - Scaling $\tilde{\mathbf{w}}$ makes $-\log_2(P(\mathbf{y}|X, \tilde{\mathbf{w}}))$ strictly smaller because $\lim_{s \rightarrow \infty} g(s \cdot t) = \mathbb{I}(t > 0)$

$$g(s \cdot t) = \frac{\exp(s \cdot t)}{1 + \exp(s \cdot t)}$$



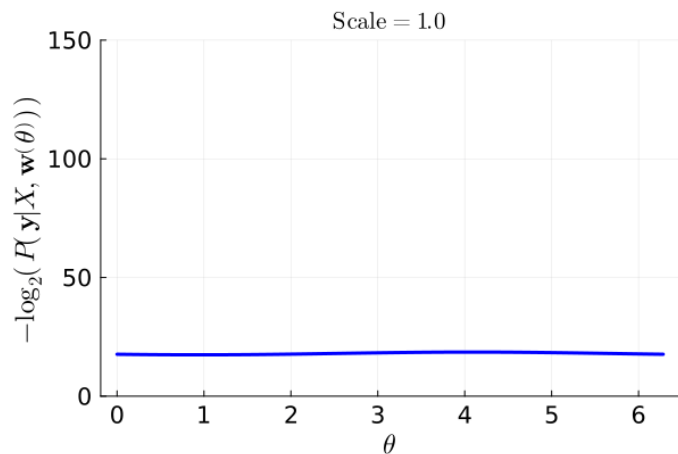
**Introduction to
Probabilistic Machine
Learning**

Unit 11 – Non-Bayesian
Classification

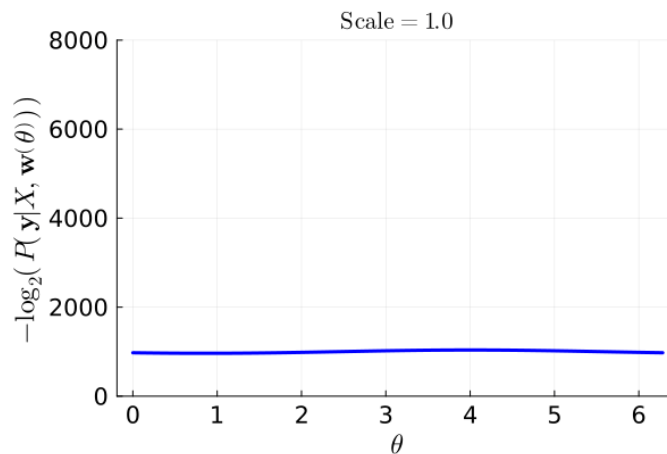
Negative Log-Logistic Likelihood in Action

$$\mathbf{w}(\theta) = \begin{bmatrix} s \cdot \sin(\theta) \\ s \cdot \cos(\theta) \end{bmatrix}$$

$n = 18$



$n = 1000$



**Introduction to
Probabilistic Machine
Learning**

*Unit 11 – Non-Bayesian
Classification*

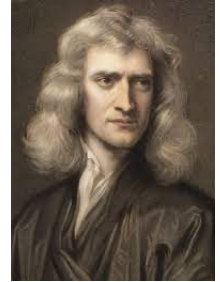
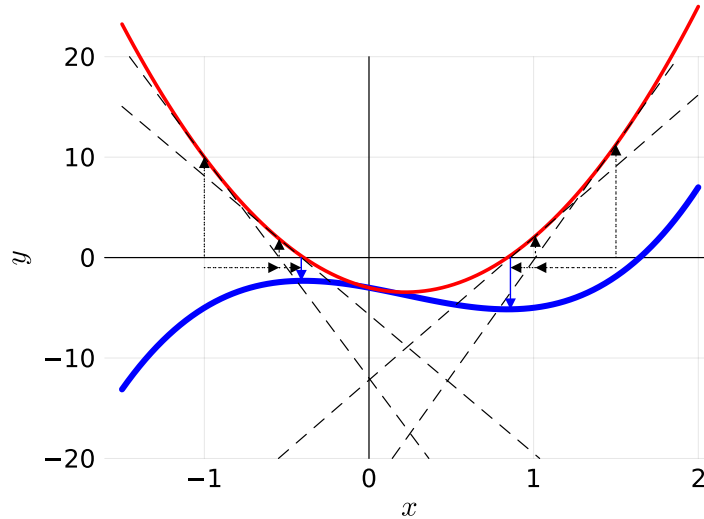
Newton-Raphson Algorithm

- **Problem:** Find the local extrema of a function $f: \mathbb{R} \rightarrow \mathbb{R}$
- **Idea:** Find the zeros of the first derivative f' of the function!
- **Newton-Raphson Algorithm:** Approximate f' at a point x_t with a linear function $g(x) = ax + b$ and find the update x_{t+1} such that $g(x_{t+1}) = 0$ via

$$a = f''(x_t)$$

$$b = f'(x_t) - f''(x_t) \cdot x_t$$

$$x_{t+1} = -\frac{b}{a} = \frac{f''(x_t) \cdot x_t - f'(x_t)}{f''(x_t)}$$



Sir Isaac Newton
(1643 – 1727)

Introduction to
Probabilistic Machine
Learning

Unit 11 – Non-Bayesian
Classification

$$x_{t+1} = x_t - \frac{f'(x_t)}{f''(x_t)}$$

Newton-Raphson in Many Dimensions

- **Problem:** Find the local extrema of a function $f: \mathbb{R}^M \rightarrow \mathbb{R}$
- **Generalized Newton-Raphson:** Start with an initial $\mathbf{w}_0$ and iterate

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \mathbf{H}^{-1}(\mathbf{w}_t) \cdot \frac{\partial f(\mathbf{w}_t)}{\partial \mathbf{w}_t}, \quad H_{i,j}(\mathbf{w}) = \frac{\partial^2 f(\mathbf{w})}{\partial w_i \partial w_j}$$

- The matrix $\mathbf{H}$ is called *Hessian matrix* (first discovered by Ludwig Otto Hesse)
- **Example:** Linear regression $f(\mathbf{w}) = \sum_{i=1}^n (\boldsymbol{\phi}(x_i)^T \mathbf{w} - y_i)^2$

$$\begin{aligned} \frac{\partial f(\mathbf{w})}{\partial \mathbf{w}} &= 2 \cdot \sum_{i=1}^n (\boldsymbol{\phi}(x_i)^T \mathbf{w} - y_i) \boldsymbol{\phi}(x_i) \\ &= 2 \cdot (\boldsymbol{\Phi}^T \boldsymbol{\Phi} \mathbf{w} - \boldsymbol{\Phi}^T \mathbf{y}) \\ \mathbf{H}(\mathbf{w}) &= 2 \cdot \boldsymbol{\Phi}^T \boldsymbol{\Phi} \end{aligned}$$

Thus, if we start with $\mathbf{w}_0 = \mathbf{0}$ and note that the Hessian does not depend on $\mathbf{w}$

$$\mathbf{w}_1 = \mathbf{0} - (\boldsymbol{\Phi}^T \boldsymbol{\Phi})^{-1} \cdot (\boldsymbol{\Phi}^T \boldsymbol{\Phi} \mathbf{0} - \boldsymbol{\Phi}^T \mathbf{y}) = (\boldsymbol{\Phi}^T \boldsymbol{\Phi})^{-1} \boldsymbol{\Phi}^T \mathbf{y}$$



Ludwig Otto Hesse
(1811 – 1874)

Matrix Derivatives

$$\frac{\partial \mathbf{A}\mathbf{x} + \mathbf{b}}{\partial \mathbf{x}} = \mathbf{A}^T$$

Newton-Raphson for Logistic Regression: Gradient

- **Negative log-Logistic Likelihood $L(\mathbf{w})$:**

$$L(\mathbf{w}) = - \sum_{i=1}^n \left[y_i \log_2 g(\boldsymbol{\phi}(x_i)^T \mathbf{w}) + (1 - y_i) \log_2 (1 - g(\boldsymbol{\phi}(x_i)^T \mathbf{w})) \right]$$

- **Gradient $\partial L(\mathbf{w}) / \partial \mathbf{w}$:** Split into two cases depending on y_i

- **Case $y_i = 1$:**

$$\frac{\partial y_i \log_2 g(\boldsymbol{\phi}(x_i)^T \mathbf{w})}{\partial \mathbf{w}} = \frac{g(\boldsymbol{\phi}(x_i)^T \mathbf{w}) \cdot [1 - g(\boldsymbol{\phi}(x_i)^T \mathbf{w})]}{g(\boldsymbol{\phi}(x_i)^T \mathbf{w})} \cdot \boldsymbol{\phi}(x_i)$$

- **Case $y_i = 0$:**

$$\frac{\partial (1 - y_i) \log_2 (1 - g(\boldsymbol{\phi}(x_i)^T \mathbf{w}))}{\partial \mathbf{w}} = - \frac{g(\boldsymbol{\phi}(x_i)^T \mathbf{w}) \cdot [1 - g(\boldsymbol{\phi}(x_i)^T \mathbf{w})]}{1 - g(\boldsymbol{\phi}(x_i)^T \mathbf{w})} \cdot \boldsymbol{\phi}(x_i)$$

- Putting it all **together** and noticing that we can use y_i

$$\frac{\partial L(\mathbf{w})}{\partial \mathbf{w}} = \sum_{i=1}^n (g(\boldsymbol{\phi}(x_i)^T \mathbf{w}) - y_i) \cdot \boldsymbol{\phi}(x_i) = \boldsymbol{\Phi}^T (\mathbf{g}(\mathbf{w}) - \mathbf{y})$$

$$\begin{bmatrix} g(\boldsymbol{\phi}(x_1)^T \mathbf{w}) \\ \vdots \\ g(\boldsymbol{\phi}(x_n)^T \mathbf{w}) \end{bmatrix}$$

Sigmoid Derivative

$$\frac{dg(t)}{dt} = g(t) \cdot [1 - g(t)]$$

Matrix Derivatives

$$\frac{\partial A\mathbf{x} + \mathbf{b}}{\partial \mathbf{x}} = A^T$$

Introduction to Probabilistic Machine Learning

Unit 11 – Non-Bayesian Classification

Newton-Raphson for Logistic Regression: Hessian

- The **Hessian** is the derivative of the first derivative

$$\frac{\partial^2 L(\mathbf{w})}{\partial \mathbf{w} \partial \mathbf{w}} = \frac{\partial \sum_{i=1}^n (g(\boldsymbol{\phi}(x_i)^T \mathbf{w}) - y_i) \cdot \boldsymbol{\phi}(x_i)}{\partial \mathbf{w}} = \frac{\partial \sum_{i=1}^n g(\boldsymbol{\phi}(x_i)^T \mathbf{w}) \cdot \boldsymbol{\phi}(x_i)}{\partial \mathbf{w}}$$

- Using **chain rule of differentiation** again, we have

$$\frac{\partial^2 L(\mathbf{w})}{\partial \mathbf{w} \partial \mathbf{w}} = \sum_{i=1}^n \frac{\partial g(\boldsymbol{\phi}(x_i)^T \mathbf{w})}{\partial \mathbf{w}} \cdot \boldsymbol{\phi}^T(x_i)$$

$$\frac{\partial^2 L(\mathbf{w})}{\partial \mathbf{w} \partial \mathbf{w}} = \sum_{i=1}^n g_i(\mathbf{w}) \cdot [1 - g_i(\mathbf{w})] \cdot \boldsymbol{\phi}(x_i) \boldsymbol{\phi}^T(x_i)$$

$$\frac{\partial^2 L(\mathbf{w})}{\partial \mathbf{w} \partial \mathbf{w}} = \underbrace{[\boldsymbol{\phi}(x_1) \quad \dots \quad \boldsymbol{\phi}(x_n)]}_{\boldsymbol{\Phi}^T} \underbrace{\begin{bmatrix} g_1(\mathbf{w}) \cdot [1 - g_1(\mathbf{w})] & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & g_n(\mathbf{w}) \cdot [1 - g_n(\mathbf{w})] \end{bmatrix}}_{R(\mathbf{w})} \underbrace{\begin{bmatrix} \boldsymbol{\phi}^T(x_1) \\ \vdots \\ \boldsymbol{\phi}^T(x_n) \end{bmatrix}}_{\boldsymbol{\Phi}}$$

- Putting it all **together**, we have

$$\frac{\partial^2 L(\mathbf{w})}{\partial \mathbf{w} \partial \mathbf{w}} = \boldsymbol{\Phi}^T \cdot R(\mathbf{w}) \cdot \boldsymbol{\Phi}$$

Sigmoid Derivative

$$\frac{dg(t)}{dt} = g(t) \cdot [1 - g(t)]$$

Matrix Derivatives

$$\frac{\partial A\mathbf{x} + \mathbf{b}}{\partial \mathbf{x}} = A^T$$

Introduction to Probabilistic Machine Learning

Unit 11 – Non-Bayesian Classification

Logistic Regression: Iterative Reweighted Least Square

- **Given:** A data set $\{(x_1, y_1), \dots, (x_n, y_n)\} \subseteq (\mathcal{X} \times \{0,1\})$ and a basis function mapping $\phi: \mathcal{X} \rightarrow \mathbb{R}^M$

1. **Initialize:** A weight vector $\mathbf{w} = \mathbf{0}$ and compute $\Phi = [\phi_j(x_i)]_{i,j=1}^{n,M} \in \mathbb{R}^{n \times M}$

2. **Iterate** until convergence

□ Compute $\mathbf{t} = \Phi \mathbf{w}$ and $g_i = \frac{\exp(t_i)}{1 + \exp(t_i)}$ and $\mathbf{R} = \text{diag} \left(\begin{bmatrix} g_1(1 - g_1) \\ \vdots \\ g_n(1 - g_n) \end{bmatrix} \right)$

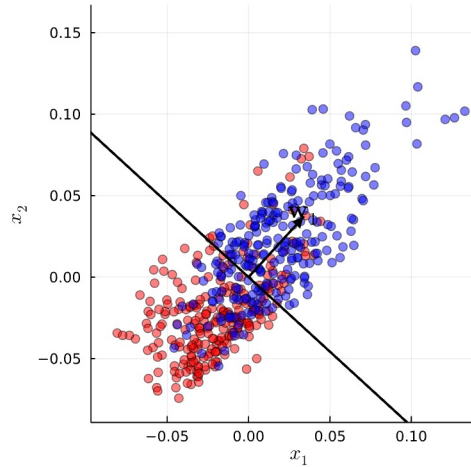
□ Update $\mathbf{w} \leftarrow \mathbf{w} - (\Phi^T \mathbf{R} \Phi)^{-1} \Phi^T (\mathbf{g} - \mathbf{y})$

Introduction to
Probabilistic Machine
Learning

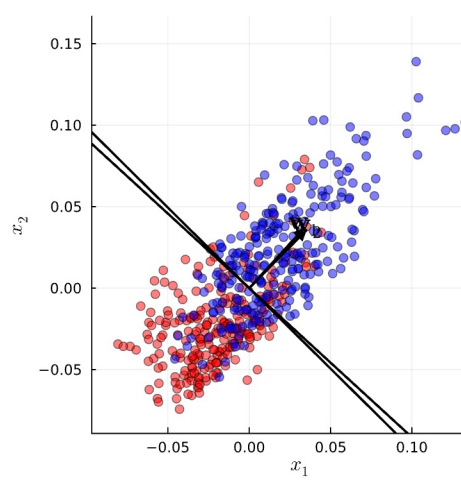
Unit 11 – Non-Bayesian
Classification

Logistic Regression in Action

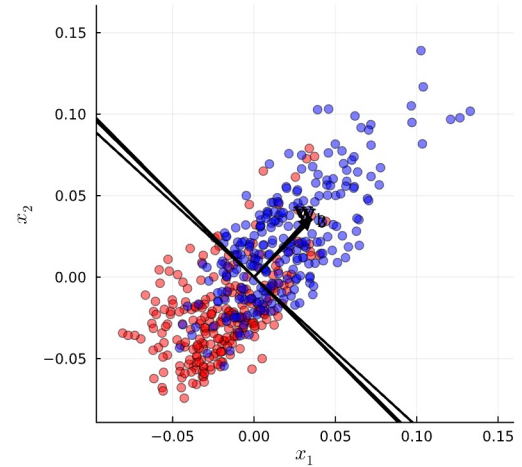
Iteration 1



Iteration 2



Iteration 3



1. Classification

- The likelihood model is no longer conjugate to the prior over the weight vector
- A linear classifier has an interpretable geometry in high-dimensional spaces

2. Maximum Likelihood: Fisher's Linear Discriminant

- Rooted in the idea of finding a projection direction where inter-class distances are maximized while intra-class distances are minimized
- Relatively simple and efficient to compute!

3. Perceptron Learning Algorithm

- *The* first classification learning algorithm which is still very relevant and performant!
- Elegant geometric convergence proof which bounds the number of update steps

4. Logistic Regression

- Assumes a logistic likelihood model (rather than a probit)
- Iterative optimization procedure using gradient descent

See you next week!